

Find The Order

Find The Order

Bài toán sử dụng audio và các pretrained model tương đối lớn. Nếu máy cá nhân không có GPU phù hợp, thí sinh được khuyến khích sử dụng Google Colab hoặc Kaggle để phát triển và thử nghiệm solution.

Dataset & Baseline: [Download Dataset & Baseline](#)

1. Bối cảnh & Mục tiêu

Trong các hệ thống xử lý ngôn ngữ nói, một đoạn hội thoại thường được tạo thành từ nhiều lượt nói của các nhân vật khác nhau. Thứ tự của các lượt nói quyết định ý nghĩa và tính mạch lạc của toàn bộ cuộc hội thoại.

Trong bài toán này, bạn được cung cấp các đoạn hội thoại tiếng Anh giữa hai người tham gia, **Speaker A** và **Speaker B**. Mỗi lượt nói được lưu thành một file âm thanh `.wav` riêng biệt.

Tuy nhiên, các lượt nói đã bị **xáo trộn ngẫu nhiên**. Tệp file `chunk_{k}.wav` chỉ biểu thị chỉ số của chunk trong tập dữ liệu đã xáo trộn, không cho biết vị trí thực sự của chunk trong cuộc hội thoại.

Nhiệm vụ của bạn là khôi phục lại thứ tự thời gian ban đầu của toàn bộ cuộc hội thoại.

2. Dataset

Mỗi dialogue gồm `n` file âm thanh:

```
chunk_0.wav
chunk_1.wav
...
chunk_{n-1}.wav
```

Mỗi chunk chứa đúng một lượt nói của một trong hai speaker. Các chunk đã được xáo trộn và có độ dài từ **7 đến 20 lượt**.

Các file âm thanh có đặc điểm:

- Mono
- Tần số lấy mẫu 44.1 kHz
- Có thể resample nếu cần thiết

Thông tin về điểm bắt đầu

File `prefix.json` cung cấp hai chunk đầu tiên theo đúng thứ tự của cuộc hội thoại.

Ví dụ:

```
{
  "11": [7, 12]
}
```

có nghĩa là với dialogue `11`:

```
chunk_7.wav -> chunk_12.wav -> ...
```

là hai lượt đầu tiên của cuộc hội thoại.

Thông tin này xác định điểm bắt đầu và loại bỏ sự mơ hồ giữa việc đọc cuộc hội thoại theo chiều thuận hoặc chiều ngược.

3. Dataset Structure

Dataset được chia thành các tập phục vụ cho những mục đích khác nhau:

Thư mục	answers.json	Mục đích
<code>dataset/train/</code>	✓	Training / fine-tuning
<code>dataset/pretrain/</code>	✓	Pretraining / training bổ sung
<code>dataset/pretest/</code>	✓	Validation / local testing
<code>dataset/test_private/</code>	✗	Final inference và tạo submission

Cấu trúc thư mục

```
dataset/
├── train/
│   ├── answers.json
│   ├── prefix.json
│   └── <dialogue_id>/
│       ├── chunk_0.wav
│       ├── ...
│       └── chunk_{n-1}.wav
├── pretrain/
│   ├── answers.json
│   ├── prefix.json
│   └── <dialogue_id>/
│       ├── chunk_0.wav
│       ├── ...
│       └── chunk_{n-1}.wav
├── pretest/
│   ├── answers.json
│   ├── prefix.json
│   └── <dialogue_id>/
│       ├── chunk_0.wav
│       ├── ...
│       └── chunk_{n-1}.wav
└── test_private/
    ├── prefix.json
    ├── <dialogue_id>/
    │   ├── chunk_0.wav
    │   ├── ...
    │   └── chunk_{n-1}.wav
```

Trong đó:

- `prefix.json` chứa thông tin về **hai chunk đầu tiên** của mỗi dialogue.
- `answers.json` chứa thứ tự đúng của các chunk.
- `answers.json` **không được cung cấp** trong `test_private`.

4. Nhiệm vụ

Với mỗi dialogue, hãy xác định vị trí theo thứ tự thời gian của từng chunk.

Kết quả của một dialogue phải là một permutation `P` của:

```
{0, 1, ..., n-1}
```

trong đó:

```
P[i]
```

là vị trí theo thứ tự thời gian của `chunk_i.wav`.

Quy ước:

- `0`: chunk đầu tiên
- `1`: chunk thứ hai
- ...
- `n-1`: chunk cuối cùng

Ví dụ

Một dialogue có 3 chunk:

Chunk	Nội dung	Vị trí đúng
<code>chunk_0.wav</code>	"No worries — I'll send you the notes afterwards."	2
<code>chunk_1.wav</code>	"Hey, are you coming to the three o'clock meeting?"	0
<code>chunk_2.wav</code>	"I can't — I've got a dentist appointment then."	1

Thứ tự đúng là:

```
chunk_1 -> chunk_2 -> chunk_0
```

Do đó:

```
[2, 0, 1]
```

là đáp án cho dialogue này.

`prefix.json` tương ứng sẽ chứa:

```
[1, 2]
```

5. Output

Bạn cần tạo một file JSON chứa dự đoán cho toàn bộ dialogue trong:

```
dataset/test_private/
```

Ví dụ:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Trong đó:

- Key là `dialogue_id`.
- Value là permutation dự đoán cho dialogue tương ứng.

6. Submission Format

Bạn phải nộp một file `.zip` duy nhất.

Ví dụ:

```
submission.zip
```

Bên trong ZIP bắt buộc phải có cả source code và prediction:

```
submission.zip
├── solution.ipynb
└── submission.json
```

`solution.ipynb`

`solution.ipynb` là source code của bạn và phải chứa toàn bộ logic cần thiết để tạo prediction.

Ban tổ chức sẽ kiểm tra sự tồn tại của file `solution.ipynb` trong submission.

Notebook nên có khả năng chạy độc lập trong môi trường thi, sử dụng dataset được cung cấp và các model được phép.

`submission.json`

`submission.json` chứa prediction cho toàn bộ dialogue trong `dataset/test_private/`.

Ví dụ:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Tên của file JSON được yêu cầu là `submission.json`.

Cấu trúc bắt buộc

Submission hợp lệ phải có ít nhất:

```
submission.zip
├── solution.ipynb
└── submission.json
```

⚠ Không nộp notebook hoặc JSON riêng lẻ. Hãy đóng gói cả hai file vào `submission.zip`.

⚠ `solution.ipynb` được yêu cầu để đảm bảo submission có source code và có thể được kiểm tra/reproduce.

7. Quy tắc của Prediction

`P` phải là một permutation hợp lệ:

- Độ dài đúng bằng `n`.
- Mỗi giá trị xuất hiện đúng một lần.
- Các giá trị nằm trong `[0, n-1]`.
- Sử dụng chỉ số 0-based.
- Không được bỏ sót dialogue nào.

Nếu permutation không hợp lệ, dialogue đó được **0 điểm**.

Dialogue bị thiếu khỏi `submission.json`, file JSON không hợp lệ hoặc dữ liệu output bị malformed cũng được tính **0 điểm** cho dialogue tương ứng.

8. Scoring

Bài toán được chấm bằng pairwise ordering accuracy.

Với mỗi cặp chunk, hệ thống kiểm tra xem hai chunk đó có được đặt theo đúng thứ tự thời gian hay không.

Một dialogue có `n` chunk sẽ có:

$$M = \frac{n(n-1)}{2}$$

cặp chunk.

Gọi `I` là số cặp bị đảo thứ tự so với đáp án đúng. Điểm của dialogue là:

$$\text{score} = 1 - \frac{I}{M}$$

Do đó:

```
0 ≤ score ≤ 1
```

Điểm cuối cùng

Điểm cuối cùng là trung bình điểm của tất cả dialogue:

$$\frac{1}{N} \sum_{i=1}^N \text{score}_i$$

Trong đó `N` là số dialogue được chấm.

9. Các mô hình được phép sử dụng

Bạn chỉ được sử dụng các pretrained model sau trong cả quá trình huấn luyện và inference:

Speech representation

wav2vec 2.0

Có thể sử dụng embedding của wav2vec 2.0 để biểu diễn nội dung âm thanh.

Automatic Speech Recognition

OpenAI Whisper, với bất kỳ kỹ thức model nào đã được cung cấp trong môi trường.

Whisper có thể được sử dụng để chuyển speech thành text và khai thác thông tin từ nội dung hội thoại.

Language model

Qwen2.5-0.5B

Có thể sử dụng:

- zero-shot; hoặc
- fine-tune trên các tập dữ liệu được cung cấp.

Các model được phép sử dụng đã được tải sẵn trong môi trường.

⚠ Không được sử dụng bất kỳ pretrained model nào khác ngoài danh sách trên.

⚠ Tổng thời gian 10 phút bao gồm cả thời gian train/fine-tune và inference.

10. Giới hạn môi trường

Toàn bộ chương trình phải chạy trong:

- Thời gian: tối đa 10 phút.
- GPU: 1 GPU, khoảng 16 GB VRAM.
- Internet: không có Internet.
- Storage: 5 GB.
- Kích thước submission: không vượt quá 1 MB.

Thời gian giới hạn bao gồm toàn bộ quá trình cần thiết để tạo kết quả cuối cùng, bao gồm:

- Khởi tạo model.
- Load dữ liệu.
- Train/fine-tune nếu có.
- Trích xuất feature.
- Inference.
- Tạo `submission.json`.

11. Local Testing

Các tập `train/`, `pretrain/` và `pretest/` đều có `answers.json`, cho phép bạn sử dụng chúng cho quá trình phát triển và kiểm tra.

Một workflow có thể là:

```
dataset/train/
├── dataset/pretrain/
│   ├── training / fine-tuning
│   └── dataset/pretest/
│       └── local evaluation
dataset/test_private/
├── final inference
├── submission.json
└── submission.zip
```

Tập `test_private/` không có `answers.json` và được sử dụng cho inference cuối cùng.

12. Cách nộp bài

- Phát triển solution bằng `dataset/train/`, `dataset/pretrain/` và `dataset/pretest/`.
- Kiểm tra solution và đảm bảo `solution.ipynb` có thể chạy đúng.
- Chạy inference trên `dataset/test_private/`.
- Tạo:

```
submission.json
```

- Đặt source code và prediction vào cùng một thư mục:

```
submission/
├── solution.ipynb
└── submission.json
```

- Nén thành:

```
submission.zip
```

- Kiểm tra submission:

```
submission.zip
├── solution.ipynb   - bắt buộc
└── submission.json  - bắt buộc
```

- Đảm bảo file ZIP không vượt quá 1 MB.
- Upload `submission.zip` lên hệ thống chấm.

⚠ Quan trọng: Submission không chỉ chứa prediction. Cả `solution.ipynb` và `submission.json` đều bắt buộc phải có trong file ZIP.

Submit Solution

Comments

Request clarification

Warning! Your default language, Python 3, is unavailable for this problem and has been deselected.

You can only submit file for this language.

Click to select a file or drag a file into this box.
Only accept ".zip". Maximum file size is 50MB.

Output Only (cat 8.32 - 9.10)

Submit!